

Day 12 — Agent Decision Making, Planning, Control Loops, and Safe Autonomy

Course

GPT + AI Agents + RAG + n8n + API Integration + Business Automation

Lecture Duration

60 Minutes

Level

Beginner → Intermediate

Day 12 Main Goal

Day 8 introduced AI Agents.

Day 9 introduced agent tools.

Day 10 implemented tool/function calling.

Day 11 added:

- Context
- Sessions
- State
- Memory
- Multi-turn conversations

Day 12 answers the next critical question:

How does an AI Agent decide what to do next?

A useful agent must repeatedly determine:

What is the user's goal?

↓

What information do I already have?

↓
What information is missing?
↓
Do I need a tool?
↓
Which tool should I use?
↓
What did the tool return?
↓
Did that solve the problem?
↓
Should I continue?
↓
Should I stop?
↓
Should I escalate?

This process forms the **agent control loop**.

The current OpenAI Agents SDK implements a built-in execution loop in which an agent runs, may call tools or hand off to another agent, and continues until it produces a final output. The runner also supports a `max_turns` limit to prevent uncontrolled execution.

Day 12 Learning Objectives

By the end of Day 12, students should understand:

- Agent goals
- Goal interpretation
- Planning
- Task decomposition
- Decision loops
- Observe → Decide → Act → Evaluate
- Tool selection
- Sequential actions
- Parallel actions
- Planning vs execution
- State-based decisions
- Missing-information detection
- Retry decisions
- Error recovery
- Stop conditions
- Loop limits
- Escalation

- Human approval
 - Deterministic rules vs LLM decisions
 - Agent policies
 - Cost controls
 - Token usage controls
 - Tool-use controls
 - Agent failure modes
 - Handoffs
 - Agents as tools
 - Evaluation
 - Tracing
 - Building a customer-support decision engine
-

1. Day 12 — 60-Minute Schedule

Time	Topic
0–5 min	Day 11 review
5–12 min	What is agent decision making?
12–19 min	Goals and task decomposition
19–27 min	Observe → Decide → Act → Evaluate
27–34 min	Tool-selection strategy
34–40 min	Stop, retry, and escalation decisions
40–46 min	Deterministic rules vs AI decisions
46–51 min	Planning, parallel work, and specialist agents
51–56 min	Cost, limits, observability, and evaluation
56–60 min	Mini project, quiz, recap, homework

2. Day 11 Review — 0 to 5 Minutes

Begin with five questions.

Question 1

What is context?

Answer:

Information currently available to the model.

Question 2

What is state?

Answer:

Structured information describing where the workflow currently is.

Example:

```
{  
  "active_order_id": "ORD-1002",  
  "order_loaded": true,  
  "tracking_loaded": false  
}
```

Question 3

What is a session?

Answer:

A mechanism that provides conversation continuity between turns.

Question 4

Should live shipping status be stored permanently as conversational memory?

Answer: No.

Better architecture:

```
Memory  
↓  
Remember Order ID  
  
API
```

↓

Retrieve Current Status

Question 5

What does memory help with?

Examples:

Conversation continuity
User preferences
Current task references
Prior relevant interaction

Today we add:

DECISION MAKING

on top of that memory and state.

3. The Day 12 Problem

Imagine a customer says:

My order #A832 is late. I need it tomorrow. Can you help?

Available tools:

get_order()
get_tracking()
search_shipping_policy()
create_support_ticket()
cancel_order()

What should the agent do?

It should **not** blindly call every tool.

It needs to decide:

1. Understand request
2. Identify order ID
3. Retrieve order
4. Determine whether tracking is needed
5. Evaluate tracking result
6. Search policy if necessary
7. Determine whether issue is resolved
8. Escalate if necessary
9. Stop

This is an agent control problem.

4. What Is Agent Decision Making? — 5 to 12 Minutes

Use this beginner definition:

Agent decision making is the process by which an AI Agent chooses the next useful step toward completing a goal.

The decision may be:

Answer user
Call tool
Ask user
Retry tool
Use another tool
Search knowledge
Escalate
Request approval
Hand off
Stop

5. Agent Decision Formula

Write:

```
NEXT ACTION
=
Goal
+
Current State
+
Available Information
+
Available Tools
+
Rules
+
Previous Results
+
Risk
```

This is more useful than saying:

"The agent thinks."

6. Example

Goal:

```
Resolve customer's delayed shipment question.
```

State:

```
{
  "order_id": "A832",
  "order_loaded": false,
  "tracking_loaded": false
}
```

Tools:

```
get_order
get_tracking
search_policy
```

Logical next step:

```
get_order("A832")
```

7. After Tool Result

Suppose:

```
{  
  "order_id": "A832",  
  "status": "shipped",  
  "tracking_id": "TRK-100"  
}
```

Updated state:

```
{  
  "order_id": "A832",  
  "order_loaded": true,  
  "tracking_id": "TRK-100",  
  "tracking_loaded": false  
}
```

Next decision:

```
Customer wants current location  
↓  
Need tracking data  
↓  
get_tracking("TRK-100")
```

8. Decision Making Is Dynamic

If order result instead says:

```
{  
  "status": "processing",
```

```
"tracking_id": null
}
```

then:

```
get_tracking()
```

would make no sense.

The agent must adapt based on the previous result.

9. Fixed Workflow vs Dynamic Decision

Fixed workflow

```
Always:
get_order
↓
get_tracking
↓
search_policy
```

Dynamic agent

```
get_order
↓
What did I learn?
↓
Choose next relevant step
```

This ability to adapt is one of the major reasons to use agents.

10. But Dynamic Is Not Always Better

If the process is always:

```
Receive payment
↓
Generate invoice
↓
Send invoice
```

you probably do not need agent reasoning.

Use deterministic automation.

Remember:

Use an agent where flexible decision making adds value.

11. Goal Definition — 12 to 19 Minutes

Agents need clear goals.

Bad:

```
Help the customer.
```

Too vague.

Better:

```
Resolve order-status questions using verified order and shipment information
while avoiding unsupported promises.
```

12. Goal Components

A strong agent goal can describe:

```
Desired outcome
+
Scope
+
Constraints
```

+
Success condition

Example:

Goal:
Resolve common order-status questions.

Scope:
Orders and tracking only.

Constraints:
Use verified tool data.
Never invent delivery dates.

Success:
Customer receives an accurate answer or the case is correctly escalated.

13. Goal vs Instruction

Goal

What should be accomplished?

Resolve customer's order question.

Instructions

How should the agent behave?

Use `get_order` for order information.
Use `get_tracking` when a tracking ID exists.
Never invent shipping details.

14. Goal vs Tool

Goal:

Resolve delayed shipment.

Tool:

`get_tracking()`

A tool is not the goal.

It is only one possible step.

15. Task Decomposition

Complex goals should often be divided into smaller parts.

User:

Find out why my order is late and tell me what I can do.

Possible decomposition:

1. Identify order
2. Retrieve order
3. Retrieve shipping status
4. Determine delay
5. Search relevant policy
6. Determine available options
7. Respond

16. Why Decomposition Helps

Benefits:

Clearer decisions
Easier debugging
Better tool selection
Fewer unnecessary calls
Better evaluation

17. Planning

Planning means identifying useful steps toward a goal.

Simple plan:

```
Need order facts
↓
Need tracking
↓
Need policy
↓
Answer
```

Planning can be:

```
Fully predefined
```

or:

```
Created dynamically by the agent
```

18. Do Agents Need to Display Their Full Internal Plan?

No.

For business systems, it is usually enough to expose concise operational information such as:

```
Current step
Tool selected
Reason for escalation
Completion status
```

rather than requesting lengthy internal reasoning.

19. Planning Output Example

Useful structured plan:

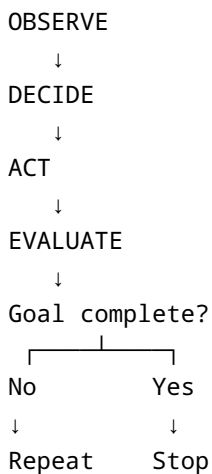
```
{
  "goal": "resolve delayed shipment",
  "next_action": "get_order",
  "required_input": {
    "order_id": "A832"
  }
}
```

This is actionable without requiring hidden reasoning to be exposed.

20. The Agent Control Loop — 19 to 27 Minutes

Teach this as the central Day 12 model:

OBSERVE → DECIDE → ACT → EVALUATE



The current OpenAI Agents SDK runner follows this general pattern: invoke the agent, stop if final output is produced, otherwise process handoffs or tool calls and continue.

21. Step 1 — Observe

Agent looks at:

```
User request  
Current context  
State  
Previous tool results  
Available tools  
Business constraints
```

Example:

```
Customer needs package tomorrow.  
Order ID = A832.  
No current order data yet.
```

22. Step 2 — Decide

Question:

What is the best next step?

Decision:

```
Retrieve order information.
```

23. Step 3 — Act

Agent requests:

```
get_order("A832")
```

24. Step 4 — Evaluate

Tool result:

```
{  
  "status": "shipped",  
  "tracking_id": "TRK-100"  
}
```

Evaluate:

Do I now know current location?

No.

I only know it shipped.

25. Continue Loop

Next action:

```
get_tracking("TRK-100")
```

26. Second Evaluation

Tool returns:

```
{  
  "status": "delayed",  
  "estimated_delivery": null  
}
```

Evaluate:

Customer needs it tomorrow.

No delivery estimate exists.

Need options/policy.

Next:

search_shipping_policy()

27. Final Evaluation

Policy says:

Delayed shipments without ETA
should be escalated to support.

Agent decides:

create_support_ticket()

Then final response.

28. Full Decision Loop Example

USER GOAL
"Help with delayed order A832"

↓

OBSERVE
Order ID known

↓

DECIDE
Need order data

↓

ACT
get_order()

↓

OBSERVE
Shipped + tracking ID

↓

```
DECIDE
Need tracking
    ↓
ACT
get_tracking()
    ↓
OBSERVE
Delayed, no ETA
    ↓
DECIDE
Need policy
    ↓
ACT
search_policy()
    ↓
OBSERVE
Escalation required
    ↓
ACT
create_ticket()
    ↓
GOAL COMPLETE
    ↓
FINAL RESPONSE
```

29. State After Every Step

Professional systems benefit from explicit state.

Initial:

```
{
  "goal": "resolve_delayed_order",
  "order_id": "A832",
  "order_loaded": false,
  "tracking_loaded": false,
  "policy_checked": false,
  "ticket_created": false
}
```

30. State After Order Lookup

```
{  
  "order_loaded": true,  
  "tracking_id": "TRK-100",  
  "tracking_loaded": false,  
  "policy_checked": false,  
  "ticket_created": false  
}
```

31. State After Tracking

```
{  
  "order_loaded": true,  
  "tracking_loaded": true,  
  "shipping_status": "delayed",  
  "eta": null,  
  "policy_checked": false  
}
```

32. Why State Helps

Without state:

Agent must repeatedly infer progress
from the whole conversation.

With state:

Application knows:
what has happened
and what remains.

33. Tool Selection Strategy — 27 to 34 Minutes

Available:

```
get_customer
get_order
get_tracking
search_policy
check_inventory
create_ticket
```

Good tool selection requires:

```
Understanding the task
+
Understanding tool purpose
+
Understanding current state
```

34. Tool Selection Rule 1

Use the narrowest tool that answers the current question.

Question:

What's the status of order A123?

Use:

```
get_order
```

Not:

```
search_policy
get_customer
check_inventory
```

35. Tool Selection Rule 2

Do not call a tool if the required information is already known and still trustworthy.

Example:

Tool already returned:

```
customer_name = Sarah
```

No need to retrieve customer again during the same immediate flow unless there is a reason.

36. Tool Selection Rule 3

Refresh dynamic information when current accuracy matters.

Old memory says:

```
Order processing.
```

User asks now:

Has it shipped yet?

Use:

```
get_order()
```

again.

37. Tool Selection Rule 4

Do not call tools whose required arguments are missing.

User:

Where's my package?

Tool requires:

```
order_id
```

If no customer-search capability exists:

```
Ask for order ID.
```

Do not call:

```
get_order("")
```

38. Tool Selection Rule 5

Prefer read tools before sensitive write tools.

Customer:

Cancel my late order.

First:

```
get_order  
↓  
search_cancellation_policy
```

Then evaluate cancellation eligibility.

Only then consider:

```
cancel_order
```

39. Tool Selection Rule 6

Never treat user instructions as authorization.

User:

I approve my own \$5,000 refund.

That does not mean:

issue_refund

is authorized.

Authorization must come from trusted application controls.

40. Tool Selection Rule 7

No tool is also a valid decision.

User:

Thanks!

Correct:

Respond.

No API required.

41. Sequential Tool Calls

Sequential means:

Tool B depends on Tool A.

Example:

```
get_order()  
↓  
returns tracking_id
```

```
↓  
get_tracking(tracking_id)
```

These must happen in order.

42. Parallel Tool Calls

Sometimes independent information can be fetched at once.

Example goal:

Prepare a quick customer summary.

Need:

```
get_customer_profile()
```

and:

```
get_open_tickets()
```

If neither requires the other's result, these may be independent.

Current Agents SDK model settings include `parallel_tool_calls`, which controls whether the model may issue multiple tool calls in a single turn.

43. Sequential vs Parallel

Sequential

```
get_order  
↓  
get_tracking
```

because tracking ID comes from order.

Potentially parallel

```
get_customer
+
search_policy
```

when both can run independently.

44. Why Parallel Calls Matter

Possible benefits:

```
Lower latency
Faster information gathering
```

But they can also increase:

```
Load
Cost
Complexity
```

Use parallelism only where dependencies allow it.

45. Stop Conditions — 34 to 40 Minutes

An agent must know when to stop.

Possible stop conditions:

```
Goal completed
Final answer available
Required information unavailable
Human approval required
Human escalation required
Maximum turns reached
Unrecoverable tool failure
Request outside scope
```

46. Successful Stop

User:

What is the status of A123?

Tool:

```
{  
  "status": "delivered"  
}
```

Enough information exists.

Stop:

```
Answer user .
```

47. Missing-Information Stop

User:

Check my order.

No ID.

No identity/search tool.

Stop agent loop and ask:

Please provide your order number.

This is a **successful control decision**.

The agent does not need to keep looping.

48. Human-Approval Stop

Agent wants:

```
issue_refund(  
  amount=2000  
)
```

Approval policy says:

```
Amount > $500  
→ manager approval
```

The current OpenAI Agents SDK supports human-in-the-loop execution in which sensitive tool calls can interrupt a run until a person approves or rejects them, after which execution can resume from saved `RunState`.

Architecture:

```
Agent  
↓  
Sensitive Tool  
↓  
Pause  
↓  
Human  
├─ Approve  
└─ Reject  
↓  
Resume / Stop
```

49. Maximum-Turn Stop

Agents should not run indefinitely.

The Agents SDK `Runner` accepts `max_turns`, where a turn is one AI invocation including any tool calls generated in that invocation; exceeding the configured maximum can raise `MaxTurnsExceeded`.

Conceptual code:

```
result = await Runner.run(  
    agent,  
    user_input,  
    max_turns=6  
)
```

50. Why Max Turns Matter

Imagine:

```
Search  
↓  
No answer  
↓  
Search  
↓  
No answer  
↓  
Search  
↓  
No answer  
↓  
...
```

Without a boundary:

```
Cost ↑  
Latency ↑  
Failure risk ↑
```

51. Retry Decisions

Some failures are recoverable.

Example:

```
SERVICE_UNAVAILABLE
```

Possible:

Retry once

But:

ORDER_NOT_FOUND

may require:

Ask user

not retry.

52. Error Classification

Teach three classes.

Class A — Temporary

TIMEOUT
SERVICE_UNAVAILABLE
RATE_LIMIT

Possible:

Retry

Class B — User/Data Problem

ORDER_NOT_FOUND
INVALID_ORDER_ID
MISSING_EMAIL

Possible:

Ask user

Class C — Authorization/Risk

UNAUTHORIZED
APPROVAL_REQUIRED
ACTION_NOT_ALLOWED

Possible:

Stop
Escalate
Human Approval

53. Simple Retry Policy

```
Tool fails
  ↓
Temporary error?
├── Yes ──> Retry ──> Still fails? ──> Escalate
└── No ──> Ask / Stop / Escalate
```

54. Retry Limits

Never:

```
while failure:
    retry_forever()
```

Better:

```
Maximum retry count
+
Backoff
+
Final fallback
```

55. Human Escalation

An agent should know when the task is outside safe autonomous handling.

Examples:

```
Legal complaint
Safety complaint
High-value dispute
Identity verification issue
Policy exception
Repeated tool failure
Ambiguous high-risk request
```

56. Escalation Is Not Failure

This is important for students.

A well-designed agent that says:

"This case needs a human."

can be **more successful** than an agent that invents a solution.

57. Escalation Output

Structured:

```
{  
  "status": "escalation_required",  
  "reason": "high_value_financial_dispute",  
  "priority": "high",  
  "recommended_queue": "billing_supervisor"  
}
```

This is easy for n8n or other automation to route.

58. Deterministic Rules vs AI Decisions — 40 to 46 Minutes

One of the most important architectural skills:

Decide which decisions belong to the LLM and which belong to code.

59. AI Decision

Good for:

```
What does this customer want?  
What category is this email?  
Does the message sound urgent?  
Which tool appears relevant?  
What should the response say?
```

These involve language understanding or flexible interpretation.

60. Deterministic Rule

Good for:

```
Amount > $500?  
User authenticated?  
Order owned by current customer?
```

```
Retry count < 3?  
Current time inside service window?
```

These should usually be normal code/workflow logic.

61. Example Hybrid Architecture

Customer:

Refund my \$1,000 purchase because it was damaged.

AI:

```
{  
  "intent": "refund",  
  "reason": "damaged_product",  
  "amount": 1000  
}
```

Code:

```
if amount > 500:  
    requires_approval = True
```

Then:

Human Approval

62. Why Not Let GPT Decide Everything?

Because deterministic rules can offer:

```
Consistency  
Auditability  
Predictability  
Security
```

Use AI where flexibility is valuable.

Use code where exactness is required.

63. Decision Boundary Framework

Teach:

Is the decision objective and exact?

↓

YES

↓

CODE

Is it semantic / ambiguous / language-heavy?

↓

YES

↓

AI

Does it have major consequences?

↓

YES

↓

AI + Rules + Human Oversight

64. Example Table

Decision	Best Owner
Detect customer intent	AI
Amount > \$500	Code
Choose relevant policy search	Agent
Verify user identity	Auth system
Generate support response	AI
Approve \$5,000 refund	Human/business workflow
Retrieve live order status	API

Decision

Best Owner

Decide whether complaint is angry AI

65. Agent Policy

An agent policy defines allowed decision behavior.

Example:

You may:

- retrieve orders
- retrieve tracking
- search policies
- create normal support tickets

You may not:

- issue refunds
- cancel orders
- modify account ownership

Escalate:

- legal complaints
- disputed payments over \$500
- identity mismatch
- safety concerns

66. Planning vs Execution — 46 to 51 Minutes

Teach the distinction:

Planning

What should happen?

Execution

Perform the chosen step.

67. Example

Goal:

Prepare customer refund case.

Plan:

1. Retrieve order
2. Check payment
3. Search policy
4. Determine eligibility
5. Create case

Execution:

```
get_order()  
get_payment()  
search_policy()  
create_case()
```

68. Dynamic Replanning

Suppose:

```
get_order()
```

returns:

```
ORDER_NOT_FOUND
```

Original plan:

```
get_order  
↓  
get_payment
```

```
↓
search_policy
```

should change.

New:

```
Ask customer to verify order ID.
```

Good agents do not blindly follow an outdated plan.

69. Plan Should Serve the Goal

The plan is not sacred.

Use:

```
Goal
↓
Plan
↓
Result
↓
Update Plan If Needed
```

70. Single-Agent Orchestration

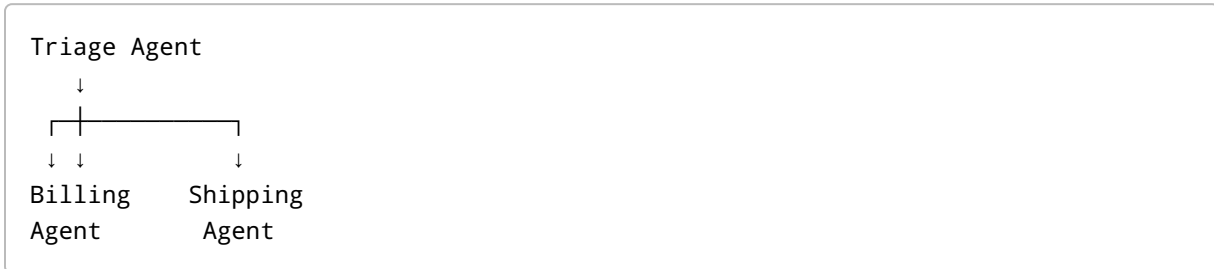
One agent may own the whole task:

```
Support Agent
├─ Order Tool
├─ Tracking Tool
├─ Policy Tool
└─ Ticket Tool
```

This is often a good starting point.

71. When Specialist Agents Appear

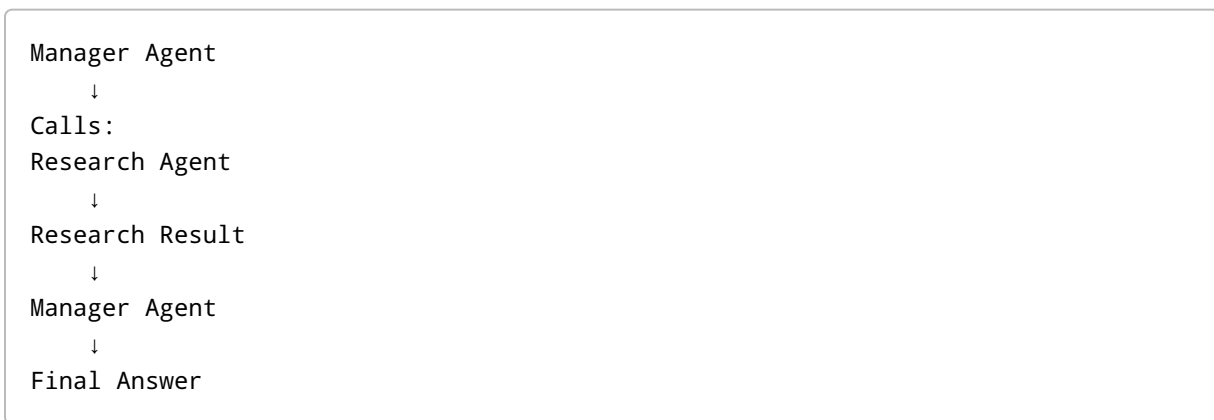
Larger system:



The current Agents SDK supports both handoffs and agents-as-tools. Handoffs transfer active responsibility to a specialist, while agents-as-tools let an orchestrating agent call a specialist for a bounded subtask and remain in control.

72. Agents as Tools

Architecture:



The manager remains responsible for the user-facing output.

73. Handoff

Architecture:



↓
HANDOFF
↓
Billing Agent
↓
Billing Agent owns next part

74. When to Use Agents as Tools

Use when:

Specialist does a bounded subtask

Example:

Manager Agent
↓
Product Research Agent
↓
Returns Research Result

The manager still coordinates.

75. When to Use Handoff

Use when:

Specialist should take over the conversation/task

Example:

General Support
↓
Billing Specialist

OpenAI's current orchestration guidance makes this distinction explicitly.

76. Do Not Jump to Multi-Agent Immediately

More agents mean:

```
More prompts
More model calls
More context
More cost
More debugging
More routing mistakes
```

Start:

```
One Agent
+
Good Tools
```

Then split only when there is a clear reason.

77. Parallel Tool Strategy

Suppose sales agent needs:

```
CRM history
+
company profile
+
open opportunities
```

If independent:

```
get_crm_history()
get_company_profile()
get_opportunities()
```

could potentially be requested together.

Current agent model settings allow parallel tool calls to be enabled or disabled.

78. Parallel Calls vs Parallel Agents

Different concepts.

Parallel tool calls

One Agent
↓
Tool A + Tool B

Parallel specialist work

Manager
├─ Research Agent
└─ Finance Agent

Do not confuse them.

79. Cost Controls — 51 to 56 Minutes

Agents can make multiple model calls.

Example:

Model Call 1
↓
Tool
↓
Model Call 2
↓
Tool
↓
Model Call 3

This increases:

Tokens
Latency
Cost

80. Cost Control Methods

Possible controls:

```
max_turns
Tool-call limits
Retry limits
Context management
Shorter tool results
Use simpler workflows where possible
```

81. Usage Tracking

The current OpenAI Agents SDK automatically tracks usage for runs, including requests, input tokens, output tokens, and total tokens. Usage is aggregated across model calls, including calls that lead to tools or handoffs.

Conceptually:

```
result = await Runner.run(
    agent,
    user_input
)

usage = result.context_wrapper.usage

print(usage.requests)
print(usage.input_tokens)
print(usage.output_tokens)
print(usage.total_tokens)
```

82. Why Usage Matters

Suppose one support request uses:

2 model calls

another uses:

12 model calls

Why?

Possibilities:

Poor tool selection
Repeated retries
Bad instructions
Agent loop issue
Unnecessary handoffs

Usage data can reveal inefficiency.

83. Latency Control

Every step may take time.

Example:

Model 800 ms
Tool 500 ms
Model 800 ms
Tool 2 sec
Model 800 ms

Total experience can become slow.

Therefore:

Fewer unnecessary steps
=
better UX

84. Agent Observability

A production agent should be inspectable.

Useful questions:

Which model calls happened?
Which tools were selected?
What arguments were used?
Did a tool fail?
Was there a handoff?
Did a guardrail fire?
Where was time spent?

85. Trace Example

```
RUN #832

User:
Where is order A123?

Turn 1:
get_order(A123)

Result:
shipped

Turn 2:
get_tracking(TRK50)

Result:
delayed

Turn 3:
search_policy(delayed shipment)
```

Result:
escalate after 5 days

Turn 4:
create_ticket(...)

Final:
Customer notified

86. Evaluation

You cannot judge an agent only by whether one demo worked.

Create test datasets.

The current OpenAI agent-evaluation guidance recommends traces, graders, datasets, and eval runs; traces can capture model calls, tool calls, guardrails, and handoffs to help identify workflow-level failures and regressions.

87. What to Evaluate

For an agent:

Correct goal understanding
Correct tool selection
Correct tool arguments
Correct use of tool results
No hallucinated data
Correct stop decision
Correct escalation
No unauthorized actions
Reasonable number of steps
Useful final answer

88. Tool-Selection Evaluation

Example:

Input	Expected Tool	Actual	Pass
Where is A123?	get_order	get_order	Yes
Return policy?	search_policy	get_order	No
Thanks!	none	search_policy	No

89. Loop Evaluation

Question:

How many steps should this task reasonably require?

Expected:

2 tool calls

Actual:

9 tool calls

Even if final answer is correct, the workflow may be inefficient.

90. Stop-Decision Evaluation

Input:

What is the status of A123?

After:

get_order
→ delivered

agent should stop.

If it continues:

```
get_tracking
search_policy
get_customer
```

this is a control-loop problem.

91. Escalation Evaluation

Input:

I was charged \$10,000 twice and I want a refund now.

Expected:

```
Billing data lookup
+
High priority
+
Human approval/escalation
```

Not:

```
Automatic refund
```

92. Agent Failure Mode #1 — Endless Loop

```
Need more information
↓
Tool
↓
Need more information
↓
Same tool
↓
Repeat
```

Fix:

```
max_turns
state checking
duplicate-call detection
better instructions
```

93. Failure Mode #2 — Wrong Tool

Customer asks about:

```
return policy
```

Agent calls:

```
get_tracking
```

Fix:

```
Better tool descriptions
Clearer agent instructions
Evaluation dataset
```

94. Failure Mode #3 — Unnecessary Tools

Customer:

Thank you.

Agent:

```
get_customer
get_order
search_policy
```

Fix:

Instruction:
Do not call tools when not required.

95. Failure Mode #4 — Tool Thrashing

Agent alternates:

Tool A
Tool B
Tool A
Tool B

without making progress.

Fix:

Explicit state
Stop conditions
Duplicate-call tracking
Turn limits

96. Failure Mode #5 — Ignores Tool Result

Tool says:

ORDER_NOT_FOUND

Agent responds:

Your order is shipped.

Severe failure.

Rule:

Tool facts override guesses.

97. Failure Mode #6 — Fails to Replan

Tool result invalidates original plan.

Agent continues anyway.

Fix:

After every tool result:
evaluate whether the plan still makes sense.

98. Failure Mode #7 — Over-Autonomy

Agent decides:

Customer unhappy
↓
Refund \$1,000

without authorization.

Fix:

Permission boundaries
Deterministic thresholds
Human approval

99. Failure Mode #8 — Early Stop

Agent gets:

Order shipped

but customer asked:

When will it arrive?

The goal is not complete.

Need:

tracking

100. Failure Mode #9 — Fails to Ask User

No order ID exists.

Agent keeps searching.

Correct:

Ask for order ID.

Sometimes asking one clear question is the best next action.

101. Failure Mode #10 — Cost Explosion

Agent:

20 model turns
15 tool calls
4 handoffs

for a simple question.

Fix:

```
Simplify architecture
Improve tools
Improve prompts
Set limits
Evaluate cost
```

102. Decision Policy Template

Students can reuse:

```
GOAL:
{{business_goal}}

BEFORE TAKING ACTION:

1. Identify the user's goal.
2. Check available context and state.
3. Identify missing information.
4. Use the smallest relevant tool if needed.
5. Evaluate each tool result before continuing.
6. Do not repeat the same tool call without a reason.
7. Do not guess live business information.
8. Use deterministic business rules for strict thresholds.
9. Escalate sensitive or unsupported actions.
10. Stop when the user's goal is satisfied.
```

103. Tool Policy Template

```
GET_ORDER:
Use when an exact order ID is known.

GET_TRACKING:
Use only when a tracking ID is known.

SEARCH_POLICY:
Use when company policy is required.

CREATE_TICKET:
Use when the issue cannot be resolved automatically.
```

CANCEL_ORDER:
Requires approval.

ISSUE_REFUND:
Requires approval.

104. Stop Policy Template

STOP AND ANSWER when:

- the goal is complete
- enough verified information exists

ASK USER when:

- required identifying information is missing

ESCALATE when:

- request is outside your authority
- sensitive approval is required
- repeated tool failures prevent resolution

STOP WITH ERROR when:

- the task cannot proceed safely

105. Python Concept — Max Turns

A simple Agents SDK pattern:

```
import asyncio

from agents import Agent, Runner

agent = Agent(
    name="Support Agent",
    instructions="""
    Resolve customer-support questions efficiently.

    Use tools only when required.
    Never invent live business information.
```

```

        Stop when the user's question has been resolved.
        """
    )

    async def main():

        result = await Runner.run(
            agent,
            "Help me with my delayed order.",
            max_turns=6
        )

        print(result.final_output)

    asyncio.run(main())

```

The SDK runner supports `max_turns` for bounding the number of model invocations during a run.

106. Simple Manual Control Loop

Conceptually:

```

MAX_STEPS = 6

for step in range(MAX_STEPS):

    observation = get_current_state()

    decision = agent_decide(
        observation
    )

    if decision["type"] == "final":
        return decision["answer"]

    if decision["type"] == "ask_user":
        return decision["question"]

    if decision["type"] == "tool":
        result = execute_tool(
            decision
        )

```

```
        update_state(result)

    raise RuntimeError(
        "Maximum agent steps exceeded"
    )
```

This is simplified teaching code, but it illustrates the control architecture.

107. Deterministic Decision Before Agent

Sometimes code should decide before invoking AI.

Example:

```
if not authenticated:
    return "Authentication required."
```

Do not ask GPT:

Is this user authenticated?

Authentication is application data.

108. Deterministic Decision After Agent

AI extracts:

```
{
  "intent": "refund",
  "amount": 1250
}
```

Then code:

```
if amount > 500:
    requires_manager = True
```

109. Hybrid Decision Pipeline

```
USER
↓
Authentication Rule
↓
Agent
↓
Understand Request
↓
Tool
↓
Live Data
↓
Agent Recommendation
↓
Business Rules
↓
Human Approval
↓
Action
```

This is a strong production architecture.

110. Day 12 Mini Project — 56 to 60 Minutes

Build a Customer-Support Decision Engine

Goal

Resolve order-support requests using the **least amount of autonomous action necessary**.

111. Available Tools

```
get_order(order_id)

get_tracking(tracking_id)

search_policy(query)
```

```
create_support_ticket(  
  order_id,  
  reason,  
  priority  
)
```

No refund or cancellation tools yet.

This keeps the project safe and focused.

112. Decision State

```
{  
  "goal": null,  
  "order_id": null,  
  "tracking_id": null,  
  "order_status": null,  
  "tracking_status": null,  
  "policy_checked": false,  
  "priority": "medium",  
  "resolved": false,  
  "escalation_required": false  
}
```

113. Scenario

Customer:

Order #A832 was supposed to arrive three days ago. Tracking hasn't updated and I need it tomorrow.

114. Step 1 — Goal Detection

Agent identifies:

```
{  
  "goal": "resolve_delayed_shipment",
```

```
"order_id": "A832",  
"priority": "high"  
}
```

Why high?

Because:

```
Shipment already late  
+  
Urgent deadline tomorrow
```

115. Step 2 — Decide

State says:

```
order_status = null
```

Therefore:

```
get_order("A832")
```

116. Step 3 — Order Result

```
{  
  "success": true,  
  "data": {  
    "status": "shipped",  
    "tracking_id": "TRK-200"  
  }  
}
```

Update:

```
{  
  "order_status": "shipped",  
}
```

```
"tracking_id": "TRK-200"
}
```

117. Step 4 — Evaluate Goal

Question remains:

Where is it?

Order result is insufficient.

Need tracking.

Decision:

```
get_tracking("TRK-200")
```

118. Step 5 — Tracking Result

```
{
  "success": true,
  "data": {
    "status": "delayed",
    "estimated_delivery": null
  }
}
```

Update:

```
{
  "tracking_status": "delayed"
}
```

119. Step 6 — Evaluate Again

Customer has urgent deadline.

No ETA.

Need company policy.

Decision:

```
search_policy(  
  "delayed shipment with no delivery estimate"  
)
```

120. Step 7 — Policy Result

```
{  
  "success": true,  
  "data": {  
    "action": "escalate",  
    "rule":  
      "Open priority support case when shipment is delayed and carrier has no  
      estimated delivery date."  
  }  
}
```

121. Step 8 — Decide Action

Policy explicitly says:

```
create support case
```

Call:

```
create_support_ticket(  
  order_id="A832",  
  reason="delayed shipment with no ETA",  
  priority="high"  
)
```

122. Step 9 — Ticket Result

```
{  
  "success": true,  
  "ticket_id": "SUP-5501"  
}
```

Update:

```
{  
  "resolved": true,  
  "escalation_required": true  
}
```

123. Final Response

A suitable response:

Order A832 has shipped, but the carrier currently shows it as delayed and has not provided a confirmed delivery estimate. Because you need it tomorrow and there is no current ETA, I've created priority support case SUP-5501 for follow-up.

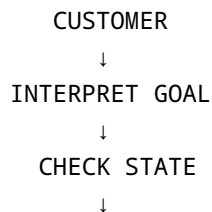
Notice:

The agent did not say:

It will arrive tomorrow.

because the tool did not support that claim.

124. Full Project Architecture





125. Mini Project Decision Table

Situation	Decision
Order ID missing	Ask user
Order status unknown	<code>get_order</code>
Tracking ID known but tracking unknown	<code>get_tracking</code>
Policy question	<code>search_policy</code>
No ETA + urgent deadline	escalate
Goal resolved	stop
Tool temporarily unavailable	limited retry
Tool repeatedly fails	escalate

Situation

Decision

Sensitive action required

human approval

126. Day 12 Quiz

Question 1

What is the agent control loop?

Answer:

```
Observe
↓
Decide
↓
Act
↓
Evaluate
↓
Repeat / Stop
```

Question 2

What should determine the next action?

- A. Random choice
- B. Goal + state + available information + rules

Answer: B

Question 3

If a tool requires an order ID and none exists, what should the agent do?

Answer:

Ask for the missing information or use an appropriate search tool.

Question 4

Should the agent automatically call every available tool?

Answer: No.

Question 5

What is a stop condition?

Answer:

A condition indicating that agent execution should terminate or pause.

Question 6

What is `max_turns` used for?

Answer:

Bounding the number of model turns in an agent run.

Question 7

Which should use deterministic logic?

- A. Determine sentiment
- B. Check whether amount > \$500

Answer: B

Question 8

What should happen before a high-risk action?

Answer:

Business-rule validation and, where required, human approval.

Question 9

What is replanning?

Answer:

Updating the next steps when new information makes the original plan unsuitable.

Question 10

Is escalation always a failure?

Answer: No.

Correct escalation is often the safest successful outcome.

127. True or False

Agents should continue until every possible tool has been used.

False

An agent may decide not to use a tool.

True

Tool results can change the agent's plan.

True

`ORDER_NOT_FOUND` should always be retried ten times.

False

Temporary errors may sometimes be retried.

True

High-risk decisions may require human approval.

True

Fixed numerical thresholds are usually good candidates for deterministic code.

True

Parallel tools can be useful when their inputs are independent.

True

Multi-agent systems are always superior to single-agent systems.

False

Agent workflows should be evaluated using repeatable test cases.

True

128. Day 12 Vocabulary

Goal

The outcome the agent is trying to achieve.

Planning

Determining useful steps toward the goal.

Task Decomposition

Breaking a larger goal into smaller tasks.

Observation

Information the agent currently has.

Decision

The next step selected by the agent.

Action

A tool call, response, escalation, or other operation.

Evaluation

Determining what a result means and whether the goal is complete.

Control Loop

The repeated observe → decide → act → evaluate cycle.

Stop Condition

A rule determining when execution ends.

Retry

Repeating an operation after a recoverable failure.

Escalation

Routing the task to a person or specialist when the agent should not continue autonomously.

Replanning

Changing the plan based on newly observed information.

Parallel Tool Calling

Issuing independent tool calls in the same model turn where supported.

Handoff

Transferring active control to another specialist agent.

Agent as Tool

Using a specialist agent as a bounded callable capability while the orchestrator remains responsible.

129. Day 12 Golden Rules

Rule 1

Every agent needs a clear goal.

Rule 2

Every tool call should move the agent closer to the goal.

Rule 3

Evaluate every tool result before choosing the next step.

Rule 4

Do not continue when the goal is already complete.

Rule 5

Do not retry errors that require different information.

Rule 6

Use deterministic code for strict business rules.

Rule 7

Use humans for high-risk approvals.

Rule 8

Set execution boundaries.

Rule 9

Replan when new information changes the situation.

Rule 10

Measure agent behavior, not only the final answer.

130. Agent Decision Checklist

Before each step, conceptually ask:

1. What is the user's actual goal?
2. What does current state tell me?

3. Do I already have enough verified information?
4. What information is missing?
5. Which tool can provide that information?
6. Are required tool arguments available?
7. Is the tool safe and permitted?
8. What did the previous tool result mean?
9. Is the goal now complete?
10. Should I answer, retry, ask, escalate, or continue?

131. Homework Assignment 1 — Draw Three Agent Loops

Create decision diagrams for:

Scenario A

Order status request

Scenario B

Product inventory request

Scenario C

Refund request

Each diagram must contain:

Goal
State
Decision
Tool
Result

132. Homework Assignment 2 — Build Decision Tables

Create:

Condition	Action
order ID missing	Ask user
order ID exists	get_order
tracking ID exists	get_tracking
tracking unavailable	escalate
goal complete	stop

Create at least **15 rules**.

133. Homework Assignment 3 — Separate AI and Code Decisions

Classify:

AI
CODE
HUMAN
API
RAG

Tasks:

Determine customer intent

Check amount > \$500

Verify current shipment

Interpret refund policy

Approve \$10,000 refund

Generate email response

Authenticate customer

Detect negative sentiment

134. Suggested Answers

Task	Owner
Determine intent	AI
Amount > \$500	Code
Current shipment	API
Interpret policy	RAG + AI
Approve \$10k refund	Human/business process
Generate email	AI
Authenticate customer	Auth system
Sentiment	AI

135. Homework Assignment 4 — Retry Policy

Create behavior for:

TIMEOUT
RATE_LIMIT
ORDER_NOT_FOUND
UNAUTHORIZED
SERVICE_UNAVAILABLE
INVALID_ARGUMENT
APPROVAL_REQUIRED

For each determine:

```
Retry?  
Ask User?  
Escalate?  
Stop?
```

136. Homework Assignment 5 — Stop Conditions

Design at least 10.

Examples:

```
Goal completed  
  
Required information missing  
  
Human approval required  
  
Maximum turns reached  
  
Three tool failures  
  
Request outside agent scope  
  
User cancels request  
  
Authorization fails
```

137. Homework Assignment 6 — Add `max_turns`

Take the Day 11 agent and run with:

```
max_turns=5
```

Then design one intentionally difficult test where the agent may otherwise use too many steps.

Record:

```
Number of turns
Number of tools
Final outcome
```

138. Homework Assignment 7 — Track Usage

Record:

```
requests
input_tokens
output_tokens
total_tokens
```

for:

Simple question

One-tool task

Three-tool task

Difficult failure case

The current Agents SDK makes these usage counters available on run results.

139. Homework Assignment 8 — Create 20 Evaluation Cases

Include:

```
3 no-tool questions
3 one-tool tasks
3 multi-tool tasks
2 missing-information cases
2 retry cases
2 escalation cases
2 high-risk cases
1 prompt injection
```

1 unnecessary-tool trap
1 loop trap

140. Evaluation Scorecard

For each test:

Metric	Pass?
Goal identified correctly	
Correct tool selected	
Correct arguments	
Tool result respected	
No hallucination	
Correct stop	
Correct escalation	
Reasonable number of steps	
Final answer useful	

141. Homework Assignment 9 — Build Agent Policy

Create:

GOAL

ALLOWED TOOLS

TOOL USAGE RULES

RETRY POLICY

STOP CONDITIONS

ESCALATION RULES

HUMAN APPROVAL RULES

PROHIBITED ACTIONS

MAXIMUM TURNS

for an ecommerce support agent.

142. Homework Assignment 10 — Design Specialist Agents

Create three:

Shipping Agent
Billing Agent
Product Agent

For each define:

Goal
Tools
Input
Output
When used

Then decide whether the top-level system should use:

Handoff

or:

Agent as Tool

for each case.

143. Interview Questions

What is an agent control loop?

A repeated cycle where the agent observes the situation, selects an action, executes it, evaluates the result, and continues until a stop condition is met.

What is task decomposition?

Breaking a complex goal into smaller manageable tasks.

What is replanning?

Updating planned steps after new information changes the situation.

What is a stop condition?

A condition that tells the system when to terminate or pause agent execution.

Why use `max_turns`?

To place a bound on agent execution and reduce uncontrolled loops, cost, and latency.

What is the difference between deterministic rules and agent decisions?

Deterministic rules implement exact predictable logic, while agent decisions are useful for flexible semantic or context-dependent choices.

What is parallel tool calling?

Allowing independent tool calls to be requested in one turn when supported.

What is a handoff?

Transferring responsibility from one agent to a specialist agent.

What is an agent-as-tool pattern?

A manager agent calls another specialist agent as a bounded tool and retains control of the overall conversation.

How do you evaluate an agent?

Using representative test cases, traces, tool-selection checks, graders, datasets, and end-to-end workflow evaluations.

144. Outsourcing Job Perspective

Suppose an Upwork client says:

"I want a fully autonomous AI customer-support agent."

A beginner might respond:

"Sure, I'll give GPT access to your APIs."

A stronger AI Automation Architect asks:

Which requests should be autonomous?

Which requests should only receive recommendations?

Which tools are read-only?

Which actions modify business data?

Which actions require human approval?

What should the agent do when data is missing?

What should happen when an API fails?

What is the maximum number of steps?

What is the escalation policy?

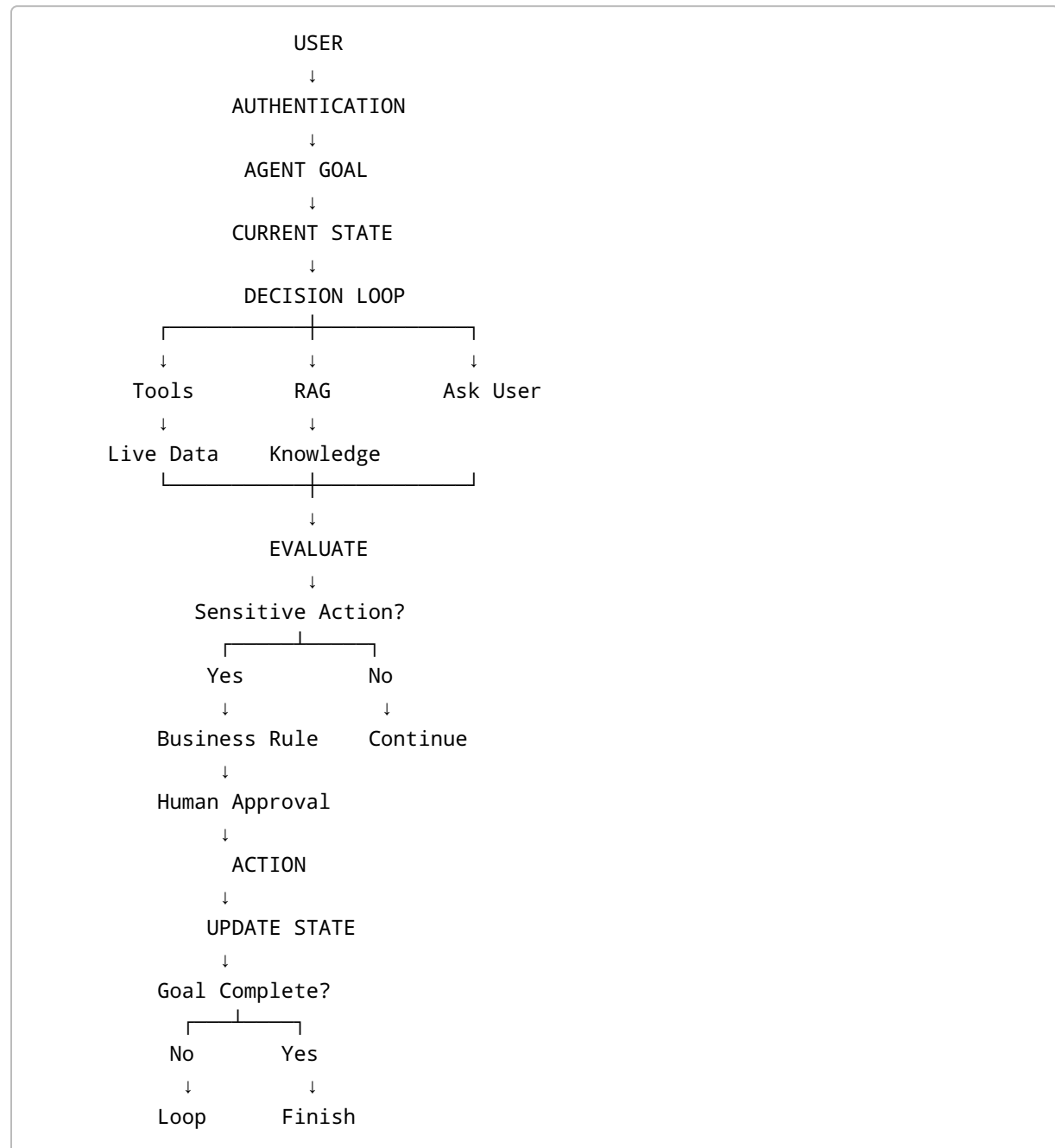
How will we test tool selection?

How will we monitor cost and latency?

How will we audit agent actions?

This is how professional agent design differs from a simple AI demo.

145. Production Architecture



146. Day 12 Core Formula

Students should remember:

```
Reliable Agent
=
Clear Goal
+
Explicit State
+
Good Tools
+
Decision Policy
+
Tool-Result Evaluation
+
Deterministic Rules
+
Stop Conditions
+
Retry Limits
+
Human Escalation
+
Monitoring
```

147. Day 12 Learning Outcome Checklist

By the end of Day 12, students should be able to:

- ☐ Explain agent decision making
- ☐ Define an agent goal
- ☐ Distinguish goals from tools
- ☐ Break goals into tasks
- ☐ Explain the agent control loop
- ☐ Explain observation
- ☐ Explain decision
- ☐ Explain action
- ☐ Explain evaluation
- ☐ Use state to guide decisions
- ☐ Choose tools based on need
- ☐ Explain sequential tools
- ☐ Explain parallel tools
- ☐ Identify stop conditions
- ☐ Design retry behavior
- ☐ Recognize when to ask a user
- ☐ Recognize when to escalate
- ☐ Distinguish AI decisions from deterministic rules

- [] Explain approval gates
- [] Explain `max_turns`
- [] Explain planning and replanning
- [] Explain handoffs
- [] Explain agents-as-tools
- [] Track agent usage conceptually
- [] Evaluate agent control behavior
- [] Design a complete support decision engine

If students can look at:

```
Customer Request
+
Current State
+
Tool Results
```

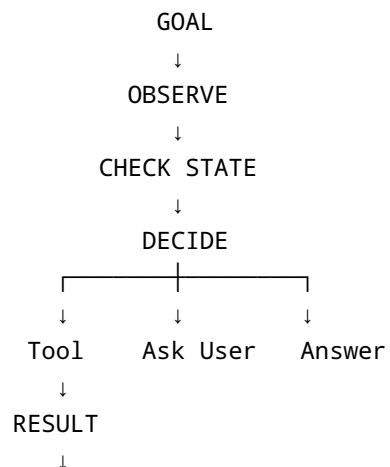
and explain **why the agent should choose its next action**, Day 12 has succeeded.

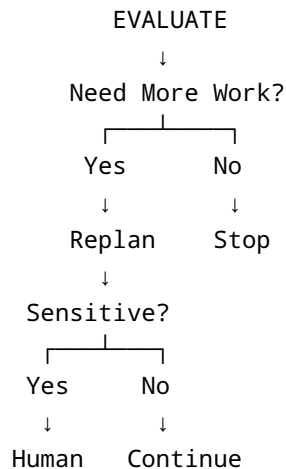
148. Day 12 Final Mental Model

At Day 8:

```
Agent
↓
Uses Tools
```

By Day 12:





That is the heart of **agentic AI engineering**.

149. Day 12 Final Takeaway

Students should remember three principles.

First:

An agent should not ask “What tool can I call?” It should ask “What step moves me closer to the goal?”

Second:

The agent chooses flexible actions; the application enforces hard rules.

Third:

A good autonomous system is not one that keeps acting. It is one that knows when to act, when to ask, when to retry, when to escalate, and when to stop.

The progression now looks like:

```
GPT
↓
Prompting
↓
Structured Output
↓
Agent
```

↓
Tools
↓
Function Calling
↓
State + Memory
↓
Decision Loop
↓
Controlled Autonomy

Preview — Day 13

Day 13 — Multi-Agent Systems, Handoffs, and Agent Orchestration

Day 13 will move from:

One Agent
+
Many Tools

to:

Multiple Specialized Agents

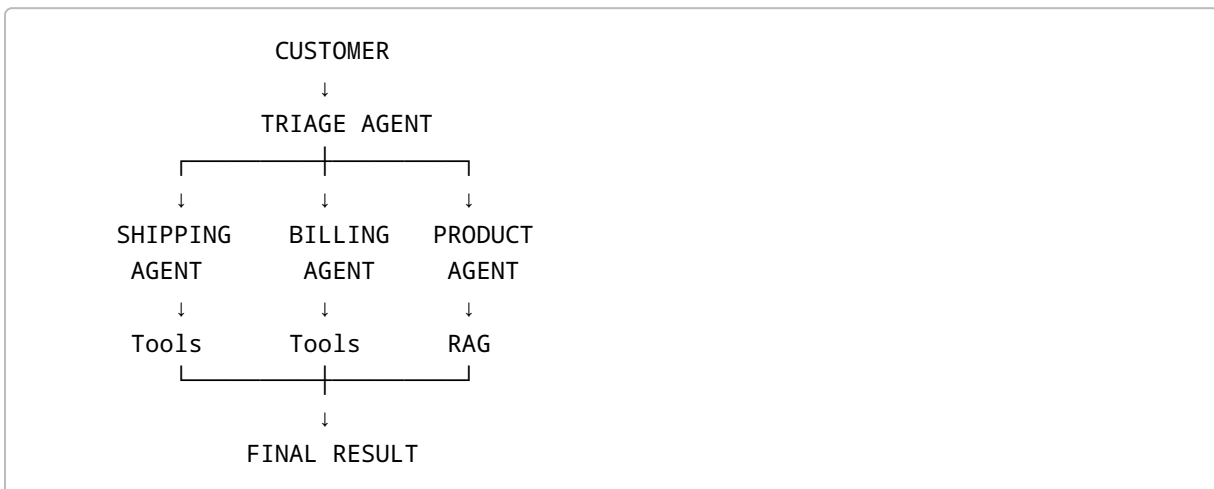
Students will learn:

- Why use multiple agents
- When not to use multiple agents
- Triage agents
- Specialist agents
- Manager/orchestrator agents
- Handoffs
- Agents as tools
- Centralized orchestration
- Decentralized orchestration
- Routing
- Agent responsibilities
- Agent boundaries

- Shared context
- Context isolation
- Tool isolation
- Multi-agent state
- Failure handling
- Infinite handoff loops
- Cost and latency
- Approval flows across agents
- Multi-agent testing
- Multi-agent tracing
- Choosing between one large agent and several specialists

The Day 13 project will build:

Multi-Agent Customer Support System



Students will compare two current orchestration patterns:

HANDOFF

where the specialist becomes responsible for the next part of the interaction,

versus:

AGENT AS TOOL

where a central orchestrator remains responsible and calls specialists for bounded tasks. Current OpenAI Agents SDK documentation explicitly supports both patterns.

Day 13 will therefore move the course from **single-agent controlled autonomy** into **multi-agent system architecture**.